

[← Go Back](#)

Hardware

Nano 33 IoT



Tutorials

Controlling a LED Through
Bluetooth® with Nano 33
IoT

Connecting Two Nano 33
IoT Boards Through I2C

**Sending an Email
Through IFTTT with Nano
33 IoT**

Accessing Accelerometer
Data on Nano 33 IoT

Accessing Gyroscope Data
on Nano 33 IoT

Nano 33 IoT Alarm with the
Arduino Cloud

Connecting Two Nano 33
IoT Boards Through UART

Connecting the Nano 33 IoT
to a Wi-Fi Network

[Home](#) / [Hardware](#) / [Nano 33 IoT](#) /

[Sending an Email Through IFTTT with Nano 33 IoT](#)

Sending an Email Through IFTTT with Nano 33 IoT

Learn how to connect the Nano 33 IoT to IFTTT to send an email each time an action is performed

Author · Fabricio Troya · Last revision · 07/17/2024

In this tutorial we will use an Arduino Nano 33 IoT, to send and email using an online automation tool that connects applications and services, named IFTTT.

Goals

The goals of this project are:

Learn what an online automation tool is.

Understand what the NINA-W102 module is.

Use the Wi-FiNINA library.

Learn how to connect Arduino to a service.

Learn how to send an email from the Arduino Cloud Editor using online automation tools.

Hardware & Software Needed

This project uses no

ON THIS PAGE

Goals

Hardware & Software Needed

Online Automation Tools

The NINA-W102 Module +

Creating the Program

Testing It Out +

Conclusion

[Help](#)

In this tutorial we will use the [Arduino Cloud Editor](#) to program the board.

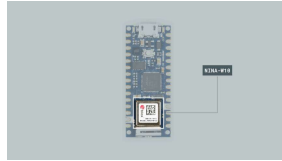
Online Automation Tools

An online automation tool is used to connect applications and services to automate repetitive tasks without having to code that process or relying on developers to build the integration. The workflows followed by these tools are "If this, do this". The first part of the process (if this), starts with a trigger, the trigger could be anything from turning on a light, to posting something on your Instagram account. The second part (do this), will run an action step of your choice, every time the set trigger event occurs. For example: if a new row is added to a google spreadsheet, then send me and email.

The NINA-W102 Module

The NINA-W102 module is a stand-alone multiradio MCU modules module that integrate a powerful microcontroller (MCU) and a radio for wireless communication. The radio provides support for Wi-Fi 802.11 b/g/n in the 2.4 Hz ISM band and Bluetooth® v4.2 (Bluetooth® BR/EDR and Bluetooth® low energy) communications

If you want to learn more about Wi-Fi and how it works, you can check [this link](#).



The NINA-W102 module.

The NINA-W102 includes the wireless MCU, Flash memory, crystal, and components for matching, filtering, antenna and decoupling, making it a very compact stand-alone multiradio module. The module can be used to design solutions with top grade security, thanks to integrated cryptographic hardware accelerators.

The Library

The WiFiNINA library allows us to use the Arduino Nano 33 IoT NINA-W102 module, without having to go into complicated programming. The library supports **WEP**, **WPA2 Personal** and **WPA2 Enterprise** encryptions and it can serve as either a **server** accepting incoming connections or a **client** making outgoing ones. Some of the most common applications for the module are the following:

Internet of Things

Wi-Fi networks

**Bluetooth® and
Bluetooth® low energy
applications**

Access to laptops, mobile phones, and similar consumer devices

Medical and industrial networking

Home/building automation

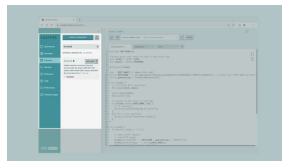
If you want to read more about the NINA-W102 module see [here](#).

In this tutorial we will use the library to connect the board to the Wi-Fi network and send a HTTP request to a specific URL to trigger an action.

Creating the Program

1. Setting up

Let's start by opening the [Arduino Cloud Editor](#), create a new sketch and name it **email_sender**. Then, navigate to the **Libraries** tab, search for the **WIFININA** library and click on the **Include** button.

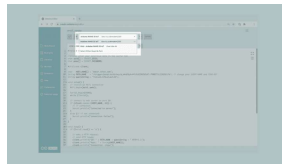


Finding the library in the Cloud Editor.

2. Connecting the board

Next, connect the Arduino Nano 33 IoT to the computer and make sure that the Cloud Editor recognizes it. If so, the board and port should appear as shown in the image below. If they don't

the Editor to recognize your board.

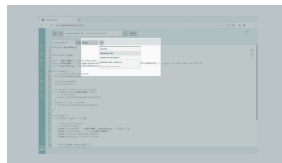


Selecting the board.

3. Sending the HTTP request

After including the library, we can begin by building the code.

Let's start by creating a secret header file to store our credential for the Wi-Fi network which we will connect to. In this way we don't accidentally store our credentials in a code we may share on the internet. To do this, we need to click the downward arrow next to our sketch tab, and click **Add Secret Tab**. This will create a tab called **Secret**.



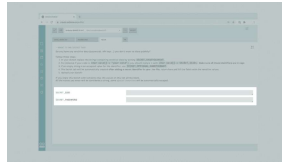
Adding a secret tab.

We now need to head back to the original sketch file, and enter the following code:

```
1 //please enter your sen
2 char ssid[] = SECRET_SS
3 char pass[] = SECRET_PA
```

This will automatically create two

back to this tab, we can enter our credentials there.



Entering the
credentials.

Now that our network credentials are safe, we can go back to the sketch and continue coding. We need to create a client that can connect to a specified internet IP address and port, using the `WiFiClient` function. We want to keep the code as simple as we possible, so in this case the name of the client will just be `client`, but you can name it however you please.

```
1 WiFiClient client;
```

Then, we need to initialize three new variables called `HOST_NAME[]`, `PATH_NAME` and `queryString` that will store the URL of the webpage we are sending the HTTP request to and the path we need to follow to trigger the action.

```
1 char HOST_NAME[] = "m
2 String PATH_NAME = "/"
3 String queryString = "?
< >
```

Note: Keep in mind that the `PATH_NAME` will be updated once we create the action on the online automation tool.

In the `setup()`, we need to start connecting to Wi-Fi using a `WiFi.begin()` function and then we start the serial communication at 9600 bauds, followed by `while(!Serial)`, which basically means that unless we open the Serial Monitor, the program will not run. We then use an `if else` statement to check if the connection to the web server was successful or not.

```
1 void setup() {
2   // initialize WiFi c
3   WiFi.begin(ssid, pas
4
5   Serial.begin(9600);
6   while (!Serial);
7
8   // connect to web se
9   if (client.connect(H
10    // if connected:
11    Serial.println("Co
12  }
13  else { // if not conn
14    Serial.println("co
15  }
16 }
```

Inside the `loop()`, we will be adding an `if()` function to check if the data enter in the Serial Monitor matches. If the data matches, we start making a HTTP request using some `.println()` functions.

```
1 void loop() {  
2   if (Serial.read() ==  
3  
4     // make a HTTP requ  
5     // send HTTP header  
6     client.println("GET  
7     client.println("Hos  
8     client.println("Con  
9     client.println(); /
```

We then add a function that will create a loop where while the client is connected (

```
while (client.connected()) ),
```

if the client have any data

available for reading (

```
if (client.available()) ), will
```

be read and print in the Serial Monitor.

```
1 while (client.connected  
2   if (client.availa  
3     // read an inco  
4     char c = client  
5     Serial.print(c)  
6   }  
7 }
```

Lastly, we add a `.stop()`

function that will stop the

communication with the client,

when the client is not connected,

and we use a `Serial.println()`

function to print that the client has disconnected.


```
1 // the server's disconn
2   client.stop();
3   Serial.println();
4   Serial.println("dis
5 }
6 }
```

4. Complete code

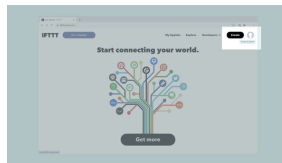
If you choose to skip the code building section, the complete code can be found below:

```
1  #include <WiFiNINA.h>
2
3  //please enter your s
4  char ssid[] = SECRET_
5  char pass[] = SECRET_
6
7  WiFiClient client;
8
9  char  HOST_NAME[] =
10 String PATH_NAME =
11 String queryString =
12
13 void setup() {
14   // initialize WiFi
15   WiFi.begin(ssid, pa
16
17   Serial.begin(9600);
18   while (!Serial);
19
20   // connect to web s
21   if (client.connect(
22     // if connected:
23     Serial.println("C
24   }
25   else { // if not cor
26     Serial.println("c
27   }
28 }
```

Testing It Out

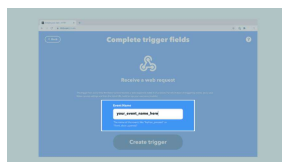
using an online automation tool. In this case, we will use the **IFTTT** (If This Then That) as our online automation tool, but you can use any tool you prefer. The process will be similar.

The first step is to log in, if you don't have an account you can create one. Then we need to click on the **Create** button on the top right. We will see the following:



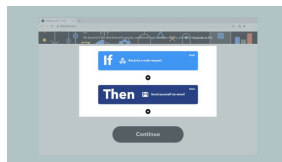
Click on the IFTTT "Create" button.

Now we need to click the **Add** button to add the trigger. We will see that we can choose a great amount of services to create a trigger, each of them consisting of different triggers. The one we need for this tutorial is called **Webhooks**, this services will activate the trigger every time the Maker services receives a web request. Click on **Receive a web request** and set an event name. We have set **sketch_received** as event name. Once we have chosen an event name click the **Create trigger** button.



Enter the webhooks event name.

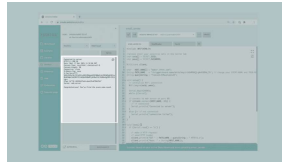
The next part is to create an action that will happen every time Webhooks receives a web request. Click the **Add** button on the **Then That** section. As before we can choose between a lot of applications and services, but the one we are going to use here is **Gmail**. The action will be to receive and email in your own account, so we need to click on the **Send yourself an email** action. Then we just need to set the subject, the body and if we want, attach an URL. Once we have set all the info we just need to click the **Create action** and then **Continue**. It should looks like the image below.



If Webhooks, then
Gmail.

Once we have created the trigger and the action, we can change the title of the applet and click the **Finish** button. The last step is to click on the **Webhooks** symbol and on the **Documentation** button in the top right. Here we can see the **Key**. We need to copy this key, go to the sketch we created before and paste the key to the variable `PATH_NAME` in the space for `your-key`. Then we need to change the `event-name` to the event name we choose in our sketch before, in this case `sketch_received`. Lastly, we just need to upload the code to the

Everything is ready! Now we just need to open the Serial Monitor, to see if the client is connected and whenever we enter an and email will be send to your own account and we will see the trigger has been activated successfully in the Serial Monitor.



Serial Monitor output.

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

- Missing a bracket or a semicolon.

- Arduino board connected to the wrong port.

- We haven't opened the Serial Monitor to initialize the program.

- We haven't changed the event name or the key

Conclusion

In this tutorial, we have learned how to simply connect to a Wi-Fi network by using the credentials in the code and send a HTTP request. We also learned how to use an online automation tool to use the HTTP request as a trigger that will execute an action on a different application or service.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

